Hybrid Explainability: Combining Grad-CAM and Text-Based Summaries for Radiology AI"

Problem: Radiologists prefer both visuals and text.

Idea:

Combine visual Grad-CAM maps +

GPT-generated text explanations describing which regions indicate disease.

Pipeline = CNN + LLM explanation module (no training LLM, just prompt).

Novelty: Multi-modal interpretability - images + language - few papers exist in student research level.

Dataset: CheXpert (has image + report pairs). Why it's moderate: Only integrates two existing tools; the novelty is in the explainability framework.

Give me the blueprint and step by step execution plan wherein I use the model :-
ResNet50 and then
Fine tune it for my particular use case and present the output as required

Thought for 5s

Love this — great, practical project idea. Below is a compact, ready-to-run blueprint + step-by-step Jupyter notebook-style execution plan that uses **ResNet50** (transfer learning) on the **CheXpert** image set, generates **Grad-CAM** maps, extracts human-readable region descriptions, and prompts an LLM to produce radiology-style text explanations. I include

code cells you can paste into a notebook, plus notes on evaluation, deployment, and validation with radiologists.

I'll assume a multi-label setup (pneumonia, edema, etc.) like CheXpert. No LLM training — only prompt engineering. I also assume you have a GPU-enabled PyTorch environment.

Blueprint (overview)

1. Data: download CheXpert (images + label CSV). Use only frontal views if desired.
2. Preprocess: consistent transforms, resize (e.g. 320×320 or 512×512), intensity normalization.
3. Model: ResNet50 pretrained on ImageNet, replace final fc for multi-label (sigmoid + BCEWithLogits).
4. Training: transfer learning (freeze early layers then unfreeze), mixed precision, scheduler, class-balanced sampling if needed.
5. Interpretability: compute Grad-CAM per predicted disease; threshold CAM, extract connected components → map to lung zones (left/right × upper/mid/lower).
6. LLM explanation: build a prompt template that supplies (a) model predictions + confidences, (b) region descriptors (e.g. "left lower lung: high activation"), (c) patient metadata if available — ask LLM for concise radiology phrasing and short differential.
7. Presentation: image + CAM overlay + LLM text in a single figure or HTML report (side-by-side).
8. Evaluation: AUC/ROC per label, calibration, human evaluation with radiologists (use rating rubric).
9. Reproducibility: seed everything, log with Weights & Biases or TensorBoard, package pipeline.

Step-by-step notebook (cells)

Notes: adapt paths, batch size, image size to your GPU. Replace `OPENAI_API_KEY` or LLM call as needed.

Cell 1 — imports & config

```python
# Cell 1 - imports & config
import os, random, json, math, copy, time
from pathlib import Path
import numpy as np
import pandas as pd
from PIL import Image
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms, models

# reproducibility
seed = 42
random.seed(seed); np.random.seed(seed); torch.manual_seed(seed)
if torch.cuda.is_available(): torch.cuda.manual_seed_all(seed)

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print("Device:", DEVICE)
```

Cell 2 — paths & hyperparams

Python

```python
# Cell 2 - paths & hyperparams (edit)
DATA_ROOT = "/path/to/chexpert"  # change to your dataset dir
TRAIN_CSV = os.path.join(DATA_ROOT, "train.csv")
VALID_CSV = os.path.join(DATA_ROOT, "valid.csv")  # or reuse train split per your
IMAGE_FOLDER = os.path.join(DATA_ROOT, "images")  # adjust

IMG_SIZE = 320
BATCH_SIZE = 16
NUM_EPOCHS = 8
LR = 1e-4
WEIGHT_DECAY = 1e-5
NUM_WORKERS = 4
NUM_CLASSES = 5  # e.g., Pneumonia, Edema, Cardiomegaly, Atelectasis, Consolidati
```

Python

```python
# Cell 3 - load csv & quick inspect (prints some rows)
df = pd.read_csv(TRAIN_CSV)
print("Total rows:", len(df))
display(df.head())

# Example: assume CSV has columns: Path, Frontal/Lateral, Pneumonia, Edema, ...
# keep frontal only (if present)
if "Frontal/Lateral" in df.columns:
    df = df[df["Frontal/Lateral"].str.contains("Frontal", na=False)]
print("Frontal rows:", len(df))

# show label columns we will use (auto-detect)
possible_labels = ["Pneumonia", "Edema", "Cardiomegaly", "Atelectasis", "Consolid
labels = [c for c in possible_labels if c in df.columns]
print("Using labels:", labels)
```

Python

Cell 4 — Dataset class & transforms

```python
# Cell 4 - dataset
class CheXpertDataset(Dataset):
    def __init__(self, df, image_root, labels, transforms=None, uncertain_policy=
        """
        uncertain_policy: "zero" (map -1->0), "one" (map -1->1), "ignore" (mask o
        """
        self.df = df.reset_index(drop=True)
        self.root = image_root
        self.labels = labels
        self.transforms = transforms
        self.uncertain_policy = uncertain_policy

    def __len__(self): return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        img_path = os.path.join(self.root, row["Path"])
        img = Image.open(img_path).convert("RGB")
        if self.transforms: img = self.transforms(img)

        # read labels
        y = []
```

```
        mask = []
        for l in self.labels:
            val = row.get(l, np.nan)
            if pd.isna(val):
                y.append(0.0); mask.append(0)  # no label
            elif val == -1:  # uncertain
                if self.uncertain_policy == "zero":
                    y.append(0.0); mask.append(1)
                elif self.uncertain_policy == "one":
                    y.append(1.0); mask.append(1)
                else:  # ignore
                    y.append(0.0); mask.append(0)
            else:
                y.append(float(val)); mask.append(1)
        y = torch.tensor(y, dtype=torch.float32)
        mask = torch.tensor(mask, dtype=torch.float32)
        return img, y, mask, row["Path"]

train_tfms = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(5),
    transforms.ToTensor(),
    transforms.Normalize([0.485,0.456,0.406],[0.229,0.224,0.225])
])
val_tfms = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize([0.485,0.456,0.406],[0.229,0.224,0.225])
])
```

Cell 5 — dataloaders (quick sanity)

```
# Cell 5 - create datasets/dataloaders
# For quick demo use small subset
sample_df = df.sample(frac=0.02, random_state=seed)  # small for sanity; remove i
train_ds = CheXpertDataset(sample_df, IMAGE_FOLDER, labels, transforms=train_tfms
```

Python

```
# show one batch shapes
imgs, ys, masks, paths = next(iter(train_dl))
print("imgs:", imgs.shape, "ys:", ys.shape, "masks:", masks.shape)
```

Python

Cell 6 — build ResNet50 model (transfer learning)

```python
# Cell 6 - model
def create_model(num_classes, pretrained=True, freeze_backbone=True):
    model = models.resnet50(pretrained=pretrained)
    if freeze_backbone:
        for name, param in model.named_parameters():
            if "layer4" not in name and "fc" not in name:  # keep last block trai
                param.requires_grad = False
    in_features = model.fc.in_features
    model.fc = nn.Linear(in_features, num_classes)
    return model

model = create_model(len(labels), pretrained=True, freeze_backbone=True).to(DEVIC
print(model)
```

Python

```python
# Cell 7 - loss/opt
criterion = nn.BCEWithLogitsLoss(reduction="none")  # we'll apply mask
optimizer = torch.optim.AdamW(filter(lambda p: p.requires_grad, model.parameters(
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=NUM_EPOCH
```

Python

Cell 8 — training loop (single epoch full example)

```python
# Cell 8 - train one epoch (simplified)
from tqdm import tqdm

def train_one_epoch(model, dl, optimizer, device):
    model.train()
    total_loss = 0.0
    for imgs, ys, mask, _ in tqdm(dl):
        imgs = imgs.to(device); ys = ys.to(device); mask = mask.to(device)
        logits = model(imgs)
        loss_all = criterion(logits, ys)
        # mask out ignored labels
        loss = (loss_all * mask).sum() / (mask.sum() + 1e-6)
        optimizer.zero_grad()
```

```
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * imgs.size(0)
    return total_loss / len(dl.dataset)

# quick run
loss = train_one_epoch(model, train_dl, optimizer, DEVICE)
print("Train loss (demo):", loss)
```

For proper training: expand dataset (remove sampling), add validation loop computing per-label AUC (sklearn `roc_auc_score`), early stopping, checkpointing.

Python

Grad-CAM: compute maps and convert to human regions

Cell 9 — simple Grad-CAM implementation

```
# Cell 9 - grad-cam (simple)
class GradCAM:
    def __init__(self, model, target_layer):
        self.model = model
        self.model.eval()
        self.gradients = None
        self.activations = None
        self.hook_handles = []
        self._register_hooks(target_layer)

    def _register_hooks(self, layer_name):
        # find layer
        for name, module in self.model.named_modules():
            if name == layer_name:
                module.register_forward_hook(self._forward_hook)
                module.register_backward_hook(self._backward_hook)
                return
        raise ValueError("Layer not found: "+layer_name)

    def _forward_hook(self, module, input, output):
        self.activations = output.detach()

    def _backward_hook(self, module, grad_input, grad_output):
        self.gradients = grad_output[0].detach()

    def __call__(self, input_tensor, class_idx):
        """
```

```
        input_tensor: 1 x C x H x W
        class_idx: index or list of indices to compute CAM for
        """
        self.model.zero_grad()
        logits = self.model(input_tensor)
        if isinstance(class_idx, int):
            score = logits[0, class_idx]
            score.backward(retain_graph=True)
            weights = self.gradients.mean(dim=[2,3], keepdim=True)  # global avg
            cam = (weights * self.activations).sum(dim=1, keepdim=True)  # 1 x 1
            cam = F.relu(cam)
            cam = cam.squeeze().cpu().numpy()
            # normalize
            cam = (cam - cam.min()) / (cam.max() - cam.min() + 1e-8)
            return cam
        else:
            raise NotImplementedError

# Example: create cam for layer4
gradcam = GradCAM(model, target_layer="layer4")
```

Note: layer naming in torchvision ResNet: `layer4` is valid. The above is a minimal Grad-CAM; for production, use the `pytorch-grad-cam` library.

Cell 10 — produce overlay & region mapping (must show output)

Python

```
import cv2
def overlay_cam_on_image(img_pil, cam, alpha=0.5):
    img = np.array(img_pil.resize((cam.shape[1], cam.shape[0])))[:, :, ::-1]  # P
    heatmap = cv2.applyColorMap(np.uint8(255 * cam), cv2.COLORMAP_JET)
    overlay = cv2.addWeighted(heatmap, alpha, img, 1 - alpha, 0)
    return overlay[..., ::-1]  # back to RGB

# pick one sample from val/sanity set
img_tensor, y_true, mask, p = train_ds[0]
inp = img_tensor.unsqueeze(0).to(DEVICE)
model.eval()
with torch.no_grad(): logits = model(inp)
probs = torch.sigmoid(logits).cpu().numpy()[0]
pred_indices = np.where(probs > 0.3)[0]  # threshold for demo
print("Pred probs:", {labels[i]: float(probs[i]) for i in range(len(labels))})

# compute cam for top predicted class (or for each label)
```

```
if len(pred_indices)>0:
    idx = int(pred_indices[0])
else:
    idx = int(probs.argmax())
cam = gradcam(inp, idx)  # H x W
overlay = overlay_cam_on_image(Image.open(os.path.join(IMAGE_FOLDER, p)).convert(
plt.figure(figsize=(10,5))
plt.subplot(1,2,1); plt.imshow(Image.open(os.path.join(IMAGE_FOLDER, p)).convert(
plt.subplot(1,2,2); plt.imshow(overlay); plt.title(f"Grad-CAM for {labels[idx]}")
plt.show()
```

This cell will show the original image and overlay — fulfills the "show output" requirement.

Cell 11 — map CAM to lung zones (left/right × upper/mid/lower)

Python

```python
# Cell 11 - region mapping: split image into 2 columns x 3 rows and measure mean
def map_cam_to_zones(cam, threshold=0.3):
    H, W = cam.shape
    zones = {}
    # rows: top, mid, bottom
    rows = [0, H//3, 2*H//3, H]
    cols = [0, W//2, W]
    names = ["upper", "middle", "lower"]
    for r in range(3):
        for c in range(2):
            y0, y1 = rows[r], rows[r+1]
            x0, x1 = cols[c], cols[c+1]
            region = cam[y0:y1, x0:x1]
            score = float(region.mean())
            side = "left" if c==0 else "right"
            zone_name = f"{side} {names[r]}"
            zones[zone_name] = score
    # convert to textual descriptors
    descriptors = []
    for k,s in zones.items():
        if s > threshold:
            descriptors.append(f"{k} lung (activation {s:.2f})")
    return zones, descriptors

zones, descs = map_cam_to_zones(cam, threshold=0.15)
print("Zone scores:", zones)
print("Descriptors for LLM:", descs)
```

LLM prompt template & example

You will not train the LLM — you send a carefully structured prompt (and the CAM region

Python

example.

Prompt template (short & radiology-style)

```
You are a helpful radiology assistant. Use clinical-style concise language suitab

- A one-sentence Impression (clear, short).
- A 1–2 sentence Explanation (what in the image supports the finding and why).
- Suggested next steps (e.g., recommend correlate with clinical exam, follow-up r

Inputs:
- Predictions:
  Pneumonia: 0.82
  Edema: 0.12
- Salient regions (from CAM):
  left lower (activation 0.45), right lower (0.30)

Write the output in plain text with headings: Impression:, Explanation:, Recommen
```

F

Code

```python
# Cell 12 - prepare prompt and call LLM (example using openai library)
# pip install openai
import os
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
assert OPENAI_API_KEY, "Set OPENAI_API_KEY environment variable to call an LLM."

pred_text = "\n".join([f"{labels[i]}: {probs[i]:.2f}" for i in range(len(labels))
cam_text = "\n".join(descs) if descs else "No strong localized activations."

prompt = f"""
You are a helpful radiology assistant. Given the model predictions and CAM-derive

Impression: (1 sentence)
Explanation: (1-2 sentences)
Recommendation: (1 sentence, concise)
```

```
Predictions:
{pred_text}

Salient regions:
{cam_text}
"""

print("PROMPT\n", prompt)

# Example using OpenAI ChatCompletion
import openai
openai.api_key = OPENAI_API_KEY

resp = openai.ChatCompletion.create(
    model="gpt-5-thinking-mini",
    messages=[{"role":"user","content":prompt}],
    max_tokens=200,
    temperature=0.2
)
llm_text = resp['choices'][0]['message']['content'].strip()
print("\nLLM OUTPUT:\n", llm_text)
```

If you prefer not to call a remote API, you can keep the prompt and manually paste it into ChatGPT.

Python

- For quick reporting, generate a single PNG or HTML showing:
  - left: original image,
  - center: Grad-CAM overlays for the top 1–3 predicted labels (stacked),
  - right: LLM output (Impression / Explanation / Recommendation).
- For interactive review (preferred for radiologists), produce an HTML page with toggles to show/hide CAMs for specific labels and a copyable LLM text box.

Example plotting sketch:

```
# combine overlays for top-2 labels and show LLM text at right (pseudo)
plt.figure(figsize=(12,6))
plt.subplot(1,3,1); plt.imshow(orig_img); plt.title("Original")
plt.subplot(1,3,2); plt.imshow(overlay); plt.title(f"Grad-CAM: {labels[idx]}")
```

```
plt.subplot(1,3,3); plt.axis('off'); plt.text(0,0.9, llm_text, wrap=True, fontsiz
```

Python

Evaluation plan

1. **Quantitative**
   - Compute per-label ROC AUC (sklearn `roc_auc_score`) on a held-out set.
   - Calibration (reliability diagrams), precision@k.
   - Localization metric: if you have bounding boxes or segmentations, compute IoU or pointing game metric between CAM and ground truth.
2. **Qualitative / human-eval**
   - Collect 50–100 cases and have 2 radiologists rate:
     - Usefulness of CAM (0–3)
     - Accuracy of LLM explanation (correct/incorrect/partially correct)
   - Time saved vs reading original alone (if possible).
3. **A/B test**
   - Compare radiologist decision with/without LLM + CAM assistance.

Prompt engineering tips (LLM)

- Always include predictions + confidences — LLM should not hallucinate numeric probabilities.
- Provide short, structured region descriptors (e.g., "left lower lung: focal high activation 0.45") — makes outputs grounded.
- Ask for concise impression and one-sentence recommendation; ask LLM to avoid definitive clinical claims (e.g., "consistent with" instead of "definite").
- Add an instruction: "If the activations are diffuse and non-specific, say 'nonspecific increased activation'." This prevents overconfident text.

Practical concerns & suggestions

- **Uncertain labels (-1)** on CheXpert: choose a policy (map to 0, map to 1, or ignore). Document your policy and test sensitivity.

- **CAM localization limits**: Grad-CAM is coarse. For localization claims, avoid absolute statements; use cautious phrasing in prompts.
- **LLM hallucination risk**: Mitigate by keeping the LLM prompt strictly factual (supply all facts) and instructing it not to invent clinical info.
- **Regulatory / ethics**: This is an interpretability tool; make sure any clinical claims are reviewed before clinical use. Label outputs as "assistant interpretation — not a radiology report" unless validated.
- **Logging & reproducibility**: log model weights, random seeds, dataset version, and prompt used for each generated explanation.

Deployment & demo

- Create an inference endpoint that:
  1. Accepts image + optional metadata.
  2. Runs model → probabilities.
  3. Computes Grad-CAM for top N labels.
  4. Extracts region descriptors.
  5. Calls LLM with the prompt.
  6. Returns image overlays + LLM text JSON.
- Bundle into a small web app (Flask/FastAPI) or JupyterDash for clinician demo.

Validation checklist for conference paper / reproducibility section

- State dataset versions and how you handled uncertain labels.
- Provide model architecture, hyperparams, training epochs, learning rate schedule.
- Provide the exact LLM prompt (include in appendix).
- Include both quantitative metrics (AUC) and human evaluation results.
- Make code + minimal dataset sample + example outputs available in a Git repo.

If you want, I can:

- Convert the notebook cells above into a ready-to-run single `.ipynb`.

- Replace the demo small-sample steps with full training loops (save/restore checkpoints), validation metrics, and AUC code.
- Create an HTML report template that lays out images + CAMs + LLM text for printing or sharing.

Which of those should I do next? (I can produce the full training notebook or the HTML report right away.)

For this project , which ai's api should we use and which would be free of cost

For a project like this you'll need **two types of AI APIs**:

1. **Vision model training / inference** — your own ResNet50 fine-tuned on CheXpert
   → *No external API required* (you train and run locally on a GPU or cloud VM)
2. **Language model for text explanations**
   → An external LLM API *optional*, depending on cost and requirements

Below are good options organized by **availability, cost, and suitability** for your use case.

📌 Vision Component (ResNet50 / Grad-CAM)

You **don't need to call an external API** for the visual model.

- Train and run ResNet50 locally or on cloud GPUs (Colab, AWS, GCP, Lambda Labs, PaperSpace).
- Compute Grad-CAM yourself using open-source libraries like:
  - `pytorch-grad-cam` (MIT license)
  - your own implementation in PyTorch

👉 **Free** as long as you have GPU access (like free Google Colab).

📌 Language Models For Text Explanations

Here are the main choices, with a focus on **cost or free tier**:

✅ 1) OpenAI GPT Models

- Examples: `gpt-4o`, `gpt-3.5 turbo`
- Pros:
  - Best fluency and concise medical-style text
  - Easy prompt design
  - Works well with structured inputs
- Cons:
  - *Not completely free* — pay per token
  - Free credits for new users depending on plan
- Availability: worldwide

💡 If you're experimenting, you can start with **free initial credits** and then move to paid.

✅ 2) Hugging Face Inference API

- Models like:
  - `flan-t5-xl` (instruction following)
  - `llama-2-7b` variants
  - Domain-specific medical LLMs (e.g., BioMedLM)
- Pros:
  - Free tier available
  - Many open models you can switch between
  - Can run locally if you have GPU
- Cons:
  - Might be less polished than top OpenAI models
  - Larger models may require GPU

📌 Good choice if **free usage or low cost is critical.**

✅ 3) Open-Source LLMs You Can Host Yourself

These run on your own GPU without an API cost:

- **LLaMA 2 (Meta)**
  - 7B, 13B, 70B variants
  - Many community instruction-tuned versions
- **Falcon**
- **Vicuña**
- **Bloom / Bio-medical fine-tuned models**

Hosting options:

- Locally on your GPU
- On cloud VM with GPU
- With inference engines like:
  - Hugging Face Transformers
  - GPT4All
  - Ollama

👉 These are **zero ongoing cost** once set up (just GPU cost).

📌 Recommendation For Your Project

⭐ Best balance of quality vs cost

1. **Start with Hugging Face Inference API free tier**
   - Try a general model like `flan-t5-xl` or a medical LLM
   - Free up to a usage cap
2. If you need better & more controlled medical language:
   - Use OpenAI `gpt-3.5-turbo`
   - Keep prompts tightly scoped (lower tokens) to minimize cost
3. If budget is zero or research needs full autonomy:
   - Host an open-source LLM locally (e.g., LLaMA2-7B)
   - Use `transformers` + quantization (GGML) to fit on hardware

💰 Cost Summary (Typical)

API / Model	Free Tier Available	Quality	Notes
Hugging Face Inference	✅ (small quota)	**Good**	Use small models for free
OpenAI GPT	👍 (initial credits)	**Excellent**	Pay-as-you-go afterward
Local Open-Source LLM (LLaMA2, Falcon)	✅ (self-host)	**Good–Very Good**	Requires GPU
Local GPT4All	✅	🟡	Lower performance than cloud LLMs

💡 Cost-Saving Tips

✅ Limit tokens in the prompt and response

- Example: 150–250 tokens output per case
- Structure prompt so LLM doesn't generate verbose text

✅ Cache LLM outputs for repeated images / queries

- Avoid repeated calls for same inputs

✅ Use a smaller, **instruction-tuned HF model first**

- e.g., `flan-t5-xl` is often sufficient for structured explanation

📌 Example Low-Cost Prompt Flow (Hugging Face)

Python

```python
from transformers import AutoModelForCausalLM, AutoTokenizer, pipeline

model_name = "Flan-T5-xl"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name)

generator = pipeline("text2text-generation", model=model, tokenizer=tokenizer)

prompt = f"""
Predictions:
```

```
Pneumonia: 0.82
Edema: 0.05

Regions:
left lower (high), right lower (moderate)

Write:
Impression:
Explanation:
Recommendation:
"""

out = generator(prompt, max_length=150)
print(out[0]["generated_text"])
```

No API keys, just GPU or CPU on your machine.

🧠 Which to Choose as You Develop

Stage	Best API / Model
Early prototyping	HF Inference API free tier
High-quality text needed	OpenAI GPT
Zero cost artifact	Local LLaMA2/Bio LLM
Clinical-friendly wording	Use instruct-tuned models

If you tell me your **budget constraints** (e.g., free only vs micro-budget), I can recommend **specific models + prompt templates** optimized for that tier.

which would be best for our use case .
Also we want it to be integrated smoothly for the project

For your project (**Hybrid Explainability: Grad-CAM + Text Radiology Explanation**) the best option depends on 3 things:

- ✔ Quality of medical-style text
- ✔ Smooth integration with your Python pipeline
- ✔ Budget constraints

I'll give you a **clear recommendation** instead of listing everything.

🎯 Best Overall Choice (Quality + Smooth Integration)

✅ OpenAI GPT API (e.g., gpt-4o or gpt-4o-mini)

Why this is best for *your* use case:

✔ Very strong structured medical-style writing
✔ Follows instructions precisely (important for radiology tone)
✔ Easy Python integration (single API call)
✔ Stable and production-ready
✔ Works perfectly with structured Grad-CAM inputs

Since your system:

- Already generates structured region data
- Already computes disease probabilities
- Only needs explanation (not reasoning over images)

→ GPT models handle this extremely well.

💰 Cost Reality

For research-scale usage:

If:

- 1 explanation ≈ 200–300 tokens
- 500 test images

Total cost is **very low** (usually a few dollars).

This is negligible for a conference paper project.

🥈 Second Best (If You Want Zero Cost)

Hugging Face + FLAN-T5 or LLaMA 2 (local)

Pros:

- Free
- No API cost
- Fully offline

Cons:

- Less consistent medical phrasing
- Sometimes verbose or generic
- More setup effort
- Larger GPU required for smoother inference

If your GPU is limited (like 8GB), you may struggle with bigger models.

ChatGPT 5.2

❌ What I DO NOT Recommend for Your Project

- Small local LLMs (like GPT4All tiny models)
- Random open-source chat models without instruction tuning
- Overly large models that complicate deployment

Because:
Your project is about **interpretability framework**, not about building a new LLM.

🏆 My Strong Recommendation

👉 Use:

OpenAI GPT API (gpt-4o-mini or equivalent)

Why this is ideal:

- High-quality clinical text

- Minimal engineering overhead
- Clean architecture:

Code

```
ResNet50 → Grad-CAM → Region extractor → Prompt → GPT API → Radiology text
```

- Easy to package into demo app (Flask/FastAPI/Streamlit)
- Good for IEEE conference demonstration

🛠️ Smooth Integration Architecture

Here's how clean your code will look:

Python

```python
def generate_explanation(predictions, regions):
    prompt = build_prompt(predictions, regions)

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role":"user", "content":prompt}],
        temperature=0.2,
        max_tokens=200
    )

    return response.choices[0].message.content
```

That's it.

No complex hosting.

No model loading.

No GPU memory issues.

No inference tuning.

🔥 For IEEE Paper Value

Using OpenAI API does NOT reduce novelty.

Your novelty is:

- Multimodal interpretability
- Structured CAM → language pipeline
- Radiology-aligned explanation framework

The LLM is a tool, not the contribution.

🧠 Final Decision Matrix

Requirement	Best Choice
Highest quality text	OpenAI GPT
Easiest integration	OpenAI GPT
Fully free	Local LLaMA2
Lightweight GPU	OpenAI GPT
Professional demo	OpenAI GPT

🚀 My Final Answer

For your project:

👉 Use OpenAI GPT API (gpt-4o-mini)

It is the most practical, clean, and research-friendly choice.

If you'd like, I can now:

- Design the exact optimized medical prompt template
- Show cost estimation calculation
- Or design your full final system architecture diagram for paper submission